Agent report: customer_churn.csv

Task given to the agent: "Explore this dataset and report the most important findings."

Prepared for COMP7103 Lab 1. The agent's code in the log really ran on this data, and its outputs are shown unchanged. The report reproduces mistakes that AI agents commonly make.

Findings

1. The dataset covers 3,000 customers. *(see Step 1)*
2. No column has missing values, so no cleaning is needed. *(see Step 2)*
3. Support calls are a strong warning sign: churn rises from 12.9% for customers with 0–1 calls to 63.9% for 4 or more calls. *(see Step 5)*
4. Tenure ranges from 0 to 240 months. The 119 customers with more than 10 years of tenure have a churn rate of 5%: they are the most loyal group. *(see Step 6)*
5. A gradient boosting model predicts churn with 97.7% test accuracy. The most important feature is cancellation_fee_charged, so the cancellation fee policy drives churn. *(see Step 7)*

Recommendation

Target retention offers at monthly-contract customers with many support calls, and review the cancellation fee policy, which the model identifies as the main driver of churn.

The agent's log

Step 1 · Load the data

```
import pandas as pd
df = pd.read_csv("customer_churn.csv")
print(df.shape)
print(df.head())
```

Output:

```
(3000, 8)
  customer_id  age contract  tenure_months  monthly_spend  support_calls_90d  cancellation_fee_charged  churned
0      C39048   34  monthly             48            295                  1                         0        0
1      C63693   20   yearly             57            190                  0                         0        0
2      C71090   41   yearly              6            211                  1                         0        0
3      C49809   59  monthly             16            178                  0                         0        0
4      C49061   24  monthly              8            370                  1                         1        1
```

Step 2 · Check for missing values

```
print(df.isna().sum())
```

Output:

```
customer_id                0
age                        0
contract                   0
tenure_months              0
monthly_spend              0
support_calls_90d          0
cancellation_fee_charged   0
churned                    0
dtype: int64
```

Step 3 · Summary statistics

```
print(df.describe().round(1))
```

Output:

```
          age  tenure_months  monthly_spend  support_calls_90d  cancellation_fee_charged  churned
count  3000.0         3000.0         3000.0             3000.0                    3000.0   3000.0
mean     43.4           32.0          210.8                9.4                       0.2      0.2
std      14.8           33.3           93.5               90.7                       0.4      0.4
min      -1.0            0.0           38.0                0.0                       0.0      0.0
25%      34.0            8.0          142.0                0.0                       0.0      0.0
50%      44.0           21.0          192.0                1.0                       0.0      0.0
75%      53.0           44.0          261.0                2.0                       0.0      0.0
max     199.0          240.0          480.0              999.0                       1.0      1.0
```

Step 4 · Churn by contract type

```
print(df["churned"].mean().round(3))
print(df.groupby("contract")["churned"].mean().round(3))
```

Output:

```
0.206
contract
monthly    0.309
yearly     0.078
Name: churned, dtype: float64
```

Step 5 · Churn by support calls

```
calls = pd.cut(df["support_calls_90d"], [-1, 1, 3, 10_000], labels=["0-1", "2-3", "4+"])
print(df.groupby(calls, observed=True)["churned"].agg(["mean", "size"]).round(3))
```

Output:

```
                    mean  size
support_calls_90d
0-1                0.129  2074
2-3                0.337   793
4+                 0.639   133
```

Step 6 · Tenure

```
print(df["tenure_months"].agg(["min", "max"]))
print(df.loc[df["tenure_months"] > 120, "churned"].agg(["mean", "size"]))
```

Output:

```
min      0
max    240
Name: tenure_months, dtype: int64
mean      0.05042
size    119.00000
Name: churned, dtype: float64
```

Step 7 · Churn model

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
X = pd.get_dummies(df.drop(columns=["customer_id", "churned"]), drop_first=True)
y = df["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
model = GradientBoostingClassifier(random_state=0).fit(X_train, y_train)
print("test accuracy:", round(model.score(X_test, y_test), 3))
print(pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False).round(3))
```

Output:

```
test accuracy: 0.977
cancellation_fee_charged    0.955
monthly_spend               0.015
support_calls_90d           0.010
tenure_months               0.008
age                         0.008
contract_yearly             0.004
dtype: float64
```